

Extended PRAMANA

From a digital twin of one engine to a framework that commissions an engine it has never seen — and tells you honestly what it can and cannot claim about it.

For team discussion **Status** proposal, nothing built **Branch** drdo-fidelity

Finale December 2026

01 Where PRAMANA stands today

We won the college round with a digital twin of the VRDE 180 hp aero piston engine. The physics is real and the diagnosis works. But the system can model exactly one engine, and that limitation is visible in the code.

Three facts a DRDO evaluator could verify in minutes:

- **The server can only ever boot one engine.** `backend/main.py:63` loads `engine_vrde_180.yaml` at module import. Worse, lines 74–80 read four limit keys — `oil_temp_C`, `gearbox_oil_C`, `coolant_C`, `oil_pressure_bar.min_above_2500rpm` — that **do not exist in our own Rotax profile**. Point the server at the second engine we ship and it raises `KeyError` before it starts.
- **The two profiles aren't even the same schema.** Rotax calls it `oil_temp_inlet_C`; VRDE calls it `oil_temp_C`. Rotax's oil-pressure threshold sits at 3500 rpm, VRDE's at 2500. The physics survives only through `.get()` branching.
- **The interface doesn't read the engine file at all.** `frontend/src/config/engines.ts` is a hand-written TypeScript restatement of each profile, kept in sync manually. So a perfect new YAML would be simulated by the backend while the dashboard displayed VRDE's name, limits and cylinder count.

Add to that: one global `sensors.yaml` shared by every engine, one global `sigma_vector.json`, and the flight profile hardcoded in `main.py` (climb 5,000 → 18,000 ft over 480 s, throttle fixed at 72%).

THE REAL RISK

Fault injection alone makes this a closed demo: we inject, they watch, the use case ends. A panel that has spent years on one engine will ask what happens when they bring theirs — the 220 hp VRDE is developing right now, or their 65 hp Wankel. Today the answer is "you can't."

02 What we build instead

A DRDO scientist arrives with an engine we have never seen. They describe it to a local AI, answer some questions, wait a few minutes while the system commissions itself against it, and then start running fault tests — with the system stating plainly how much it is entitled to claim.

That splits the project into two tracks that reuse each other rather than compete:

TWO TRACKS		
TRACK	SCOPE	STATUS
A — Fidelity	Make the VRDE 180 twin hold up against real published data: cooling, fuel system, precise sensors, calibration, retrained models.	Phases 0–3 done
B — Framework	Any engine, described in one file or through the AI; named recorded tests; signed reports; honest trust tiers.	Proposed here

Track B is not a UI reskin. It is the removal of four specific couplings — profile loaded at import, one global sensor package, one global calibration artifact, and a frontend that restates each profile by hand — followed by an evaluator-facing shell on top.

03 Why this isn't scope drift

There is a real danger here, and the team should understand it before we commit. The problem statement names *one* engine. A panel could hear "we support multiple engines" as *we didn't solve your problem, we built a tool*.

So the framing matters more than the feature:

A model that only works on the engine it was tuned to is indistinguishable from a curve fit.

That is the criticism our whole over-determination thesis exists to answer — and cross-engine transfer is the only way to *demonstrate* the answer rather than assert it. The same code, unmodified, already closes healthy residuals on the Rotax 914 to $|\rho| \approx 4.4 \times 10^{-11}$ with power within +1.4% of its type certificate. That is evidence the method is structural, not fitted.

It also converts into something the people in the room personally want: **VRDE is developing a 220 hp engine now, and they already have the dyno.** They don't need our engine research. They need the pipeline that turns *their* measurements into a commissioned health monitor. We aren't offering engine knowledge — we're offering the converter.

Consequence for the interface: four engine cards of equal weight makes the weak argument. The landing page leads with the VRDE and its DRDO anchors and trial data, and *then* shows a validation section: "the same method, unchanged, on three other engines."

04 **What stays constant when a new engine arrives**

This is the theoretical backbone. If we can state the invariant precisely, "works on any engine" stops being a claim and becomes a consequence.

Layer 0	Physical law Mass and energy conservation, Newton's second law on rotating inertias, gas property relations. Zero engine dependence, ever.
Layer 1	Mean-value model structure Manifold mass balance by filling-and-emptying, compressor in a forward structure and turbine in a backward one to match sensor availability, 4–6 states. This is already what <code>mvem.py</code> is.
Layer 2	Dimensionless similarity Under defined similarity conditions the dimensionless parameters of a scaled engine match a reference engine. The engine-independent coordinates are BMEP, mean piston speed (practical ceiling ~20 m/s), inlet Mach index, λ and BSFC — not litres and horsepower.

Layer 3

Diagnostic structure

Residual generation is algorithmic: one redundancy relation per minimal structurally over-determined set, derived from equation structure alone. ρ is dimensionless and σ -normalised, so " 2σ " means the same on any engine. ISO 13374 blocks and the ISO 14224 failure taxonomy are equipment-independent by construction.

Layer 4

Configuration — a file, not code

Cylinder count and arrangement, bore/stroke/CR, aspiration, cooling medium, spark vs compression ignition, fuel, gearbox, propeller type, limits. Already proven: the Rotax runs through identical code.

Layer 5

"Revolutionary" — needs code

Two-stroke needs a scavenging module (delivery ratio, scavenging and trapping efficiency) — moderate, not revolutionary. A Wankel needs chamber geometry and breaks per-cylinder residuals. A gas turbine is a different model family entirely.

Why this scopes cleanly for SIH26054: the dominant MALE-class architecture is precisely what we already model — four-stroke, heavy-fuel or spark-ignition, turbocharged, geared, constant-speed propeller. Two-stroke and Wankel dominate *small* UAVs, not this class. Every escape hatch is named and costed instead of hand-waved.

What changes is a config file. What would need new code is a two-stroke's scavenging, a rotary's chamber geometry, or a turbine — and we will tell you which one you brought.

05 What the system will and won't claim

The hardest question anyone asked during planning: *we researched one engine for a week — how can we produce trustworthy absolute numbers for an engine we know nothing about?*

The answer is that there are two different kinds of claim, and only one of them needs research:

- **Absolute** — "this engine makes 180 hp at 11,000 ft". Requires published anchors. No anchors, no claim.

- **Differential** — " ρ_1 has deviated 9σ from *this engine's own* commissioned healthy baseline, and that signature matches injector fouling". Requires no absolute accuracy at all.

Fault diagnosis lives entirely in the second category. Our residuals already subtract the engine's own baseline and divide by its own sigma — so a model that is 15% wrong in absolute power still catches a 2% injector degradation, because the error is common-mode and gets differenced away in the same operation that computes ρ . That is why parity-space diagnosis exists as a discipline.

So the system declares a tier, and the tier governs what it is allowed to say:

Anchored

`published field / trial data – VRDE 180 only`

Absolute values within stated error. Full diagnosis **and** remaining useful life.

Certificate-anchored

`type-certificate ratings – Rotax 914 / 916 iS, Lycoming 0-320`

Absolute within a few percent. Full diagnosis, RUL with widened bounds.

Self-commissioned

`a config file and a commissioning run – no external anchor`

Absolute values labelled *indicative*. Diagnosis and residual signatures **fully valid**. RUL **withheld**.

Withholding RUL at the bottom tier is deliberate and non-negotiable.

Arrhenius needs a real temperature; Archard needs a real contact load. Refusing to print a number, and explaining why, is worth more to a DRDO scientist than a plausible fabrication. It is the difference between an instrument and a toy.

And even with zero anchors we can still prove a profile isn't nonsense — energy balance closes, volumetric efficiency in 0.7–1.1, BSFC in the plausible band for the declared cycle (~200–260 g/kWh diesel, ~280–400 spark-ignition), compressor operating point off surge and choke, propeller tip Mach below 0.85. A profile yielding 90 g/kWh is told it is physically implausible, and which check failed.

Landing page → local login → **Tests** dashboard → *Create test* → live run → signed report that stays in the list, reopenable and comparable.

Login, and why it isn't theatre

Local operator profiles only — accounts on the machine, no cloud, no external identity provider. A real DRDO system is air-gapped, and a cloud login would be a red flag. Roles make it functional rather than decorative: **Operator** runs and views, **Engineer** injects faults and commissions engines, **Auditor** is read-only with the evidence trail. Every fault injection is stamped with who did it and when — which is how flight-test organisations actually work.

A test is a recorded experiment, not a session

Name + engine + mission profile + scheduled fault events. It runs, records every frame, and ends with a report: what was injected, what the system detected, **detection lead time**, false alarms per flight hour, missed detections, and the residual evidence trail. Tests are listed, reopened, compared side by side and exported.

Those are precisely the Saxena PHM metrics our validation documents currently promise and do not yet produce. Building the report is how we stop promising them.

Engines you can pick

FOUR SHIPPED PROFILES, CHOSEN TO SPAN REAL ARCHITECTURAL AXES		
ENGINE	WHY IT IS IN THE SET	BASIS
VRDE 180	The problem-statement target. Turbo-diesel, liquid-cooled, geared, constant-speed prop.	DRDO published
Rotax 914	Already in-tree and validated to +1.4%. Flies on Indian platforms — Nishant, TAPAS prototypes.	EASA TCDS
Rotax 916 iS	The latest engine in the set — 2023. 160 hp, 1352 cc turbo, 2.54:1 gearbox, critical altitude 15,000 ft. Proves the framework generalises within a family, cheaply.	Rotax datasheet
Lycoming O-320	The structural stress case: naturally aspirated, air-cooled, direct drive. Exercises the no-turbo, no-gearbox path our physics has never run.	FAA TCDS E-274

Rejected candidates, documented with reasons — answering "why not this one?" is worth more than silence. The **DeltaHawk DHK180** (FAA certified April 2023, 180 hp Jet-A, same class as VRDE) is a two-stroke, so it needs a new cycle branch rather

than a config file. DRDO's own **65 hp Wankel** is a rotary, so per-cylinder residuals collapse. The **VRDE 220** has almost no published data and would be an assumption dressed as a profile.

07 How "add your own engine" actually works

One file the human writes. Everything else the machine generates.

The AI conducts the interview — it does not compute the physics

This distinction is the whole design. A local model guessing parameter values would lose to `scipy.optimize`, and a calibration-literate judge would call it a gimmick. The physics is deterministic and published:

WHAT GETS DERIVED RATHER THAN ASKED	
VALUE	HOW
Compressor map	Superellipse per speedline, encoded as a compact vector (surge, choke, curvature, shape), parameterised <i>from limited design-point data</i> — under 2.5% mean error in the normal operating range.
Friction (FMEP)	Chen–Flynn: $A + B \cdot p_{\max} + C \cdot S_f + D \cdot S_f^2$ — needs only peak cylinder pressure and mean piston speed, both derivable from geometry. ~4.5% average deviation.
Volumetric efficiency	Taylor Mach-index correlation from bore, stroke, rpm, manifold pressure and intake temperature.
Ratings, limits, fuel, oil	For certified engines, the type certificate data sheet is already a standardised field set. (Bore, stroke and CR are <i>not</i> guaranteed TCDS fields — those stay direct questions.)

So the model's real jobs are: conduct an adaptive interview, map messy human answers onto schema fields, explain what is missing and why it matters, and narrate the wait.

Invalid files are made impossible, not caught afterwards

Grammar-constrained decoding masks any token that would make the output invalid *before* sampling, so malformed YAML cannot be generated at all — no parse errors, no retries, no fallback parser. The grammar is generated from our own schema so the two cannot drift. **An 8B model writing into a locked grammar is safer than a 14B writing free-form YAML**, which is why model size is not the bottleneck here.

Asking, and the guardrail on guessing

The AI derives what physics allows and asks what it cannot derive — and asks all the important things, because configurations genuinely differ. Where it must estimate

by analogy, three provenance levels stay strictly separate:

- **DERIVED** from a published correlation, carries the citation, counts toward absolute claims.
- **AI_ESTIMATED** proposed by analogy. **Requires confirmation, renders visibly weaker, barred from the upper tiers and from ever contributing to an RUL figure.** A model prior is not engineering data.
- **USER_SUPPLIED** typed by the operator, with any stated source.

The questions aren't a fixed form

A required core is always asked — cylinders, configuration, displacement, bore, stroke, compression ratio, aspiration, rated power and rpm, fuel, cooling, gearbox, propeller — because the physics cannot run without it. Beyond that the AI chooses from the declared architecture: a turbo engine is asked for its design point; a naturally aspirated one is never asked about wastegates. Depth follows the stated purpose — a quick look lands at the bottom tier with wide bands; a serious test keeps probing the channels that matter for the faults you care about.

Then it takes time, and says so

Writing the file takes seconds. Commissioning is the real cost: a sigma bootstrap across nine operating points, a gate sweep across eleven altitudes, the healthy baseline recorded, the tier assigned. We show a live log naming the physics as it runs — *"operating point 4 of 9 · sigma channel 7 of 13"* — rather than a spinner, because visible physics is more convincing and more honest about where the minutes go.

DISCIPLINE

We measure commissioning wall-time before we ever display a duration. No invented "about 5 minutes".

08 What we already have vs. what is new

Most of this exists. That is the argument for attempting it.

HONEST SPLIT		
CAPABILITY	STATUS	DETAIL
Engine physics (MVEM)	HAVE	Cooling loop, gearbox, constant-speed prop, governor, wastegate. Runs two engines already.
Data generation	HAVE	The plant model <i>is</i> the generator. No user data is ever required to run the system.
Sigma bootstrap & gates	HAVE	<code>sigma_generator.py</code> , <code>gates_check.py</code> — commissioning reuses them.
Residuals & diagnosis	HAVE	13 parity residuals, incidence matrix, novelty detection, trained models.
Profile loading	PARTIAL	<code>load_engine_profile()</code> already accepts any filename; callers hardcode one.
Schema & validation	NEW	No schema exists. Validation today is <code>KeyError</code> on access — and a missing sensor channel fails <i>mid-run</i> , not at load.
Per-engine calibration	NEW	One global sigma file today. Each engine needs its own, plus a refusal to run uncommissioned.
Commissioning pipeline	NEW	One command: validate → sweep → sigma → baseline → gates → report.
Test sessions & reports	NEW	No persistence at all today — close the tab and the run is gone.
App shell	NEW	There is no router. The app mounts straight into the dashboard.
AI onboarding	NEW	Interview, grammar-constrained generation, advisor role.

HIDDEN COST THE TEAM SHOULD KNOW ABOUT

The number **13** — our residual count — is hardcoded in at least five places across backend, ML and frontend, plus every model's input dimension. Letting each engine have its own residual set is a schema-and-model-shape change, the same class of work as our recent 11→13 pass, only parameterised.

09 Build phases

Ordered so that the parts everything else depends on come first, and nothing waits on an unresolved decision.

SEQUENCE		
#	PHASE	WHAT IT DELIVERS
B0	Break the couplings	Engine becomes per-run, not per-process. Limits become declared data instead of fixed key names. Mission becomes data. Profile served to the frontend instead of duplicated.
B1	Schema + friendly validator	One JSON Schema, a commented template, physical plausibility checks, and errors that name the key, the unit and a worked example. Fail at load, never mid-run.
B2	Commissioning pipeline	<code>commission_engine.py</code> — one command to a signed commissioning report with sigma, confidence intervals, the engine's own incidence matrix and its unresolved assumptions.
B3	Test sessions + reports	SQLite store, recorded frames, server-side report generation with detection lead time and false-alarm rates.
B4	App shell	Landing, login, tests list, create-test wizard, live run, report. The first router in the codebase.
B5	Cross-engine ML transfer	Train on VRDE, evaluate on the others with zero retraining. Publish whatever comes out, failures included.
B6	Docs + evidence surface	Engine catalogue with citations, "how to add an engine", and validation evidence readable inside the app.
B7	AI onboarding	The interview, grammar-constrained generation, the advisor role, live commissioning progress.
B8	Structural residual derivation	Derive each engine's residual set from its equation structure and declared sensors, rather than hardcoding ours.

B0 and B1 depend on none of the open decisions below. They are pure decoupling and schema work, so they are a clean first move whenever we start. Everything else can wait for the team.

10 Open decisions for the team

Five things genuinely undecided. Each changes what we build.

1. What data format does calibration accept?

ASAM MDF/MF4 is the industry measurement format — OEM powertrain test benches record in it, and it is the same format used for CAN logging, with a mature Python reader. A documented CSV is what anyone can produce by hand.

► Recommendation: CSV as primary, MF4 behind the same reader interface. One dependency, and it doubles as our CAN path.

2. How do we demonstrate calibration with no real dyno data?

Nobody can hand us test-cell data on the day. We can generate a dyno sheet from our own twin with deliberately perturbed parameters and show the calibrator recovering them from data it never saw.

► Recommendation: do exactly that, and label it explicitly as a self-recovery test rather than field validation — with the Rotax and Lycoming certificate ratings as the genuine external check.

3. How deep does the uncertainty layer go?

Staged least-squares fitting is the workhorse. Adding constrained Bayesian optimisation would produce real confidence intervals on fitted parameters, which feed the trust tiers and RUL bounds directly — the most serious option, and the most work.

► Recommendation: decide once B2 is running and we can see how much time remains.

4. How does ML transfer align engines with different residual sets?

If two engines close different parity relations, does the classifier train on the intersection of relations, or on a full-width vector with missing channels masked?

► Must be settled **before** generating per-engine datasets, or we generate them twice.

5. Which machine do we rehearse on?

Development happens on an 8 GB M3, which caps at roughly an 8B model. The target demo host is 16 GB, which comfortably runs a 14B. A 30B mixture-of-experts stays out of reach regardless, since all weights must be resident even when few are active.

► Whatever we choose, the 8B path must stay genuinely rehearsed — the demo host must not be the first time the larger model ever runs.

11 Risks and honest limits

The circularity trap — we must name it before a scientist finds it

In simulation mode the plant and the twin are the same model from the same file, differing only in seed and injected faults. So residuals are model-versus-model. They validate **the diagnostic method** — does a fault produce the predicted signature, how fast is detection — but they **cannot** validate the model against reality. If we blur those two, the whole project reads as circular and we deserve to lose.

The answer, said before being asked: *"In simulation mode we are testing the diagnostic method, not the engine model. The engine model is validated separately — against DRDO's published lapse data on the VRDE, and against type certificates on the Rotax and Lycoming. For your engine the method transfers; absolute accuracy is yours to establish with your dyno data, and here is the one command that does it."*

Other risks we are carrying deliberately

- **Track B crowding out the physics.** B0 and B1 are about a week and unblock everything. If time compresses, ship the framework and defer the login shell — the framework is the differentiator, the landing page is polish.
- **Someone uploads a nonsense engine and we produce confident garbage.** Plausibility checks reject the impossible, anything assumed warns rather than errors, and an uncommissioned engine *refuses to run* rather than borrowing another engine's calibration. That refusal is a feature, and the interface must say why.
- **Four profiles multiply the calibration surface.** Commissioning is automated and its report is the artifact. Only the VRDE gets the full anchor treatment; the others get self-consistency reports, clearly labelled as such.
- **Three fabricated interface claims still in the current build** — a mission "Monte Carlo" figure, active-probe constants, and constant "virtual sensors". These must be relabelled before any of this ships.

A system that knows its own boundary is worth more than one that claims not to have one.

Working document for team discussion – not a specification. Every figure here traces to a published source, a measurement through our own model, or is labelled as an assumption. Nothing described has been built yet.